

Multi-modal ship tracking and identification

First steps: Multiobject tracking



Prepared by:

Amir Dollie

Prepared for:

EEE3000X

Yaaseen Martin

Department of Electrical Engineering

University of Cape Town

January 30, 2026

Contents

1	Introduction	1
1.1	Problem Description	1
1.2	Scope and Limitations	1
1.3	Purpose of this document	2
1.3.1	Github Link	2
2	Methodology	3
2.1	Prototype Design	3
2.2	Final Design	3
3	Prototype Design	4
3.1	Ground truth acquisition	4
3.2	Detector training	4
3.3	Tracking	6
4	System Design	7
4.1	Ground truth acquisition	7
4.2	Training the detector shipDetectorV1	8
4.3	Training the detector shipDetectorV2	9
4.4	Multi-object tracking	10
5	Discussion and Results	13
5.1	A note for whoever is potentially taking this on	13
6	Resources	14
6.1	MATLAB resources	14
6.2	ML/DL resources	14
6.3	Alternate data sets/next steps	14
7	Literature Review on AIS-Image Data Fusion	16
7.1	Transformation Methods	16
7.1.1	Homography	17
7.1.2	Perspective-n-point (used only by BONK)	19
7.1.3	Interpolation (Emre Gulsoylu et al)	20
7.2	Association Methods	20
	Bibliography	21

Chapter 1

Introduction

1.1 Problem Description

Ship detection and tracking is an essential part of maritime activities to predict ship routes in order to avoid collision paths as well as to identify malicious or unauthorized vessels.

Computer vision is an effective way of tackling this problem, but it alone is an unreliable method of correctly and reliably identifying and tracking vessels. This is due to the high noise environment these detections must take place in, from major occlusions due to waves and obstacles as well as the relatively unpredictable movement of these vessels relative to the potentially moving camera which must make these detections.

The advent of data fusion helps to increase the reliability of these detections, where multiple sources of information are combined to generate a more complete picture of the scenario. Combining data such as GPS messages, radar, LIDAR and multiple cameras and correlating the information relayed from each allows for the most complete depiction of reality where a machine learning/deep learning computer vision algorithm could accurately depict ship paths and detect/identify specific ships.

1.2 Scope and Limitations

The scope of this project is to design the building block algorithm before data fusion, one which utilizes only computer vision techniques to accurately and reliably track multiple ships present in a given frame of video.

The project does not include the fusion or synthesis of multiple data sources in order to correctly identify the ships present in the video, but rather to identify and generate tracks for the ships present, associating a temporary identity to each track which will later be resolved to their true identity through the incorporation of AIS/GPS data.

The limitations for this project are listed below:

- The dataset utilized is extracted from MIT's Sea Grant AUV lab.
- Ship tracking can only be resolved through the use of video data.
- Of the three available cameras (left, right and center), only the right camera formed the training dataset.
- Only MATLAB was utilized.

1.3 Purpose of this document

This document is intended to serve as a reference/guide to whoever wishes to expand upon this project. It will outline the basics of each MATLAB file generated (which are found on the Github), as well as a quick introduction to machine learning/image processing that is necessary for this project.

1.3.1 Github Link

Below is the github link which contains the live MATLAB files created for this project, as well as some useful documents:

<https://github.com/AmirDollie/Multi-Object-Tracker-for-Marine-Vessels>

Chapter 2

Methodology

2.1 Prototype Design

Before actual implementation, a smaller scope prototype was developed, including only single object tracking. The methodology utilized here was expanded upon later to include multi-object tracking.

1. Ground truth acquisition.
2. Implementation and design of the detection algorithm.
3. Post-processing on detection outputs.
4. Tracking algorithm implementation and design.
5. Evaluation of final outputs.

2.2 Final Design

The methods and analysis of the final design for this task were refined through the evaluation of the prototype model, as well as the added complexity to fulfill the full scope of the project. Nonetheless, the methodology for this follows closely for that of the prototype:

1. Ground truth acquisition.
2. Detection algorithm design and implementation.
3. Post-processing on bounding box outputs of the detection algorithm.
4. Tracking algorithm design and implementation.
5. Tuning of tracking algorithm parameters.
6. Evaluation of final design outputs.
7. Addition of visualization of outputs.

Chapter 3

Prototype Design

3.1 Ground truth acquisition

For this prototype, the ground truth data was chosen to be an arbitrary video with a single moving vessel not present in the final design's data set.

For ground truth creation from video footage, you need it to look like the expected outputs of your algorithm. This means that each frame in the video needs to have bounding boxes (bboxes) drawn around each ship in each frame, with the correct labels associated with each (in this case there is only one class 'ship'). This was done with MATLAB's built in VideoLabeller app, where an example frame is depicted below:



Figure 3.1: Labeled ground truth data for prototype

This was repeated for each frame in the video. If you would like access to my labelled ground truth data for this particular video, please download the file from the github `gTruth_Right.mat`.

3.2 Detector training

To interactively follow along, view the `shipTracker.mlx` file in the github.

For this prototype, a simple machine learning pretrained network was utilized, the ACF object detector (aggregated channel features). This object detector was chosen due to its lack of depth (shallow ML architecture means that it won't take long to train). This algorithm was actually trained on the entire video, which is something that was improved upon in later sections. Once training was complete, it was tested on an arbitrary frame, see the 50th annotated frame below:

the high bbox score.

Now even though this postprocessing worked well for this dataset, it may not work in all cases. If detection is crucial for your application, I would recommend a few things:

1. Utilize a deeper convolutional neural network for object detection, such as a YOLOX model.
2. Shuffle the video frames so it is not overfit for the dataset.
3. Perform image augmentation techniques on the frames of the video before gTruth labelling, such as resizing, horizontal and vertical reflections and applying higher or lower contrast.

3.3 Tracking

To track the ship in every frame, every bbox was converted to their centroid (ie from its 4D position output to simple (x,y) pixel coordinates), and fed into a Kalman filter to predict the location of the next centroid.

The Kalman filter was fed pre-chosen values of measurement noise, initial error, as well as its motion model chosen to be 'constant velocity'. These are all tunable parameters! It computes a covariance matrix and then uses negative feedback between its predicted centroid and the actual measured centroid of the next frame to fine tune its own parameters, see [1]. The reason this was implemented was not for track identification, but for occlusion resistance. It works by outputting both the bbox centroid as well as the KF predicted centroid in each frame, such that when there is an occlusion, the predicted centroid is still shown. Below is a picture showing the bbox centroid (green) and the predicted centroid (red) for a frame from the video:



Figure 3.5: Final output of the prototype detector/tracker

This worked well for this video, please find the full annotated video on the github in the output folder of the Prototype model.

Chapter 4

System Design

4.1 Ground truth acquisition

Now that the prototype model had been finalized, the same pipeline could be followed for the final design. Firstly, an adequate data set needed to be identified, one which had all the relevant data required for the full scope of the underlying project.

There is a serious lack of acceptable data sets for this task. Only two were found, FVessel and the AUV lab from MIT, please find links to their corresponding papers and databases respectively [2], [3]. The FVessel data set size was very large, approximately 30GB, so the AUV lab dataset was chosen for ease of use. It also contains more data from more data sources than the FVessel data set, which only contained video data and AIS messages. The AUV lab dataset contained videos from multiple angles, GPS messages, infrared video as well as radar and lidar data.

The ground truth data was chosen to be a subset of the right hand camera's video. In this video, there are at most three ships available at any instance in time in each frame, ship 2 (blue) and ship 6 and 17 (yellow). Ships 6 and 17 are indiscernible from one another, except for their numbering (which is admittedly hard to see because of the low pixel count per frame). Please find the annotated 400th frame of gTruth below:

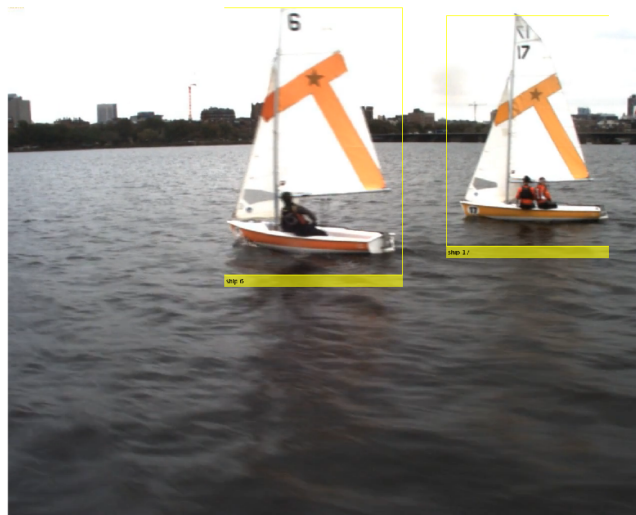


Figure 4.1: Annotated 400th frame with 2 visible classes

You can download and view the full gTruth variable from the github: gTruth_Right.mat

4.2 Training the detector shipDetectorV1

Please follow along in document a1 from the github to interact with and view the specifics of how this detector was trained.

Initially, the idea was to utilize pure vision based methods to identify and track the ships. This meant that the detector needed to identify the ships from one another, reliably and accurately.

A more sophisticated method of training the detector was utilized here, as well as a stronger pretrained network. I decided to utilize YOLOX as the detector, and applied transfer learning to ensure it was trained on the ground truth data and accept the correct image dimensions.

To ensure that the ship detector is not too overfit to the data set, I split it into 70% training, 10% validation and 20% testing. Moreover, I shuffled the frames of the video so it doesn't have preconceived notions of where ships will be in the next frame ie reducing variance.

The YOLOX model utilized was the 'tiny-coco' network, and the optimization algorithm was the adam optimizer. The adam optimizer is an optimization algorithm (necessary for all ML related tasks as the ultimate goal is to minimize a loss function) and it combines ideas from other optimizers such as stochastic gradient descent (with momentum) as well as RMS_Prop. The super basic pseudocode for how the adam optimizer works is:

```
firstM = 0;
secM = 0;

numIterations = 1E6;
for t = 1:numIterations
    dx = computeGrad(x);
    firstM = beta1 * firstM + (1-beta1)*dx;
    secM = beta2 * secM + (1-beta2)*dx^2;

    first_unbias = firstM/(beta1^t);
    second_unbias = secM/(beta2^t);
    x = x - alpha*first_unbias/ ( sqrt(sec_unbias) + 1E-7);
end
%often set beta1 = 0.9, beta2 = 0.999, alpha = 1E-3
%dynamically change alpha as t increases, either:
%cosine: alpha_t = 0.5 * alpha_0 * (1+cos(t*pi/T)) or linear or invsq etc
```

Figure 4.2: Pseudocode to help understand what the Adam optimizer does

Instead of inspecting the performance by eye per frame, it was decided that a confusion matrix for the detector over the whole training set would be more informative. The diagonal elements of the matrix correspond to correct detections, while off diagonal elements are incorrect classifications:

ship ₇	24				
ship ₂		9			
ship ₆			18		
ship ₇					
unmatched	2	1	1		
	ship ₇	ship ₂	ship ₆	ship ₇	unmatched

Figure 4.3: Confusion matrix for shipDetectorV1 on gtTest. Ship 7 is a dummy class with no ship 7 ever being present.

This worked almost perfectly for this right hand video. Upon testing this detector on the center and left cameras, it was unfortunately noted that it almost always misclassified the vessels. It would create bboxes around the relevant ships, but the labels would be incorrect, implying that it was actually overfit for the training set. Moreover, due to the similarity with ships 6 and 17, it could almost never tell the difference between the two. To give qualitative meaning to the above sentences, one could create precision-recall curves for the detector on the training set. This would help classify:

1. Whether the detector can identify if an object is a ship.
2. Whether the detector can discern between ships.

A method of increasing the probability of correct detections would include more examples in the training set, and augmenting the frames with rotations and reflections and applying contrast and so forth. But the goal of this project was not to correctly ID ships based on detections anyways, so this detector and method was removed from the final pipeline.

4.3 Training the detector shipDetectorV2

Please follow along in document a3 from the github to analyze the specifics of how this detector was trained.

Noting the failure of shipDetectorV1, a new method was proposed. Instead of having the detector detect the ID of each ship, rather have it simply classify the objects in the frame as a single class 'ship'. After which, the tracking method can be utilized for identification.

To do this, the same ground truth variable as what trained shipDetectorV1 was modified, with each class label replaced with 'ship'. Here is the modified 400th frame:

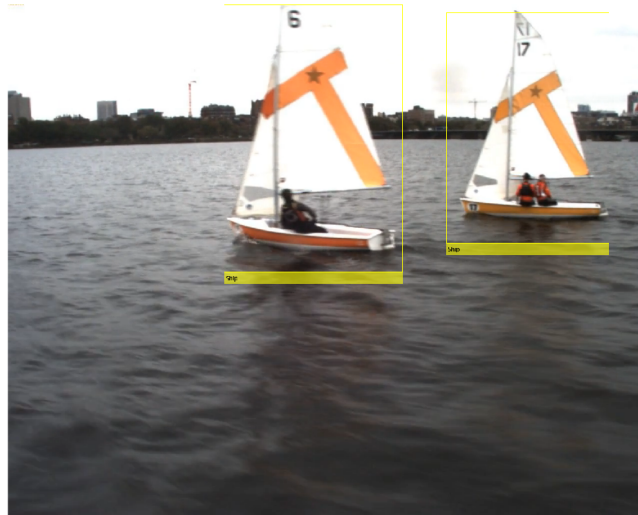


Figure 4.4: Annotated 400th frame of the modified gTruth with a single class

The same split of training, validation and testing as before was conducted, but this time the training options were modified. To remove false confidence, validation frequency was changed to occur (number of elements in training)/2 times every epoch. Minibatchsize was changed to 2 with a max number of epochs being 10. This allowed the detector to go through the shuffled dataset 10 times, resulting in the following loss minimization curve:

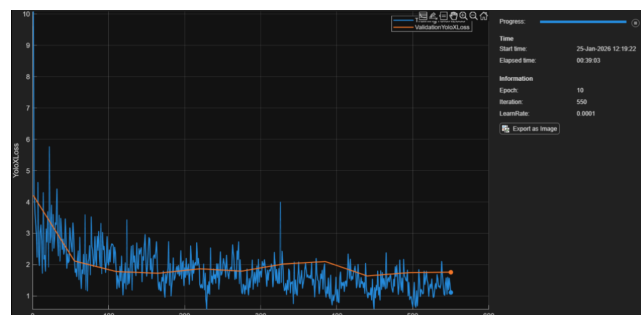


Figure 4.5: Loss curve for the detector. Blue is the loss on the training data and orange is on the validation data.

The adam solver was used again over stochastic gradient descent, along with a small learning rate of $1e-4$.

This detector detected ships correctly, such that post processing was irrelevant.

4.4 Multi-object tracking

The initial idea for multi-object tracking was to adapt the code for single object tracking and apply it to the current program. This required a Kalman filter to be initialized and used to predict the centroid of the bbox for each individual bbox. This immediately came with many issues.

Firstly, track ID and retention proved to be almost impossible with this method. The Kalman filter

was too eager to associate new tracks as opposed to retain track identity. The way it works is that each bbox is converted to its (x,y) centroid pixel location. Then the algorithm generates a cost matrix for the next predicted centroid, where the minimum distance the predicted centroid is from the previous frames centroids defines which track it belongs to.

Below is an example frame of the first implementation:



Figure 4.6: Red circles are predicted centroids, yellow boxes encompass red circles, cyan text is the track ID. Note how the ID's are already in the 30's, but these are the first 2 ships in the video, hence should have ID's 1 and 2!

The main parameters that could be tuned in this algorithm were the measurement noise and max assignment distance. Unfortunately changing either of these had little effect on the track ID retention issue.

After struggling with this, it was decided that a built in multi-object tracker would be more useful than the manual Kalman filter. To do this, trackerGNN was utilized. This algorithm uses the global nearest neighbour and greedy assignment to associate centroids with tracks. For trackerGNN documentation, please read [4]. This implementation worked well, but it still was a bit too volatile with regards to assigning a new track ID.

A solution for this issue was to only show confirmed tracks that exist for more than 10 frames. This allows a confirmation period to pass where volatile quick changes are filtered out of the pipeline. Below is an example of the final trackers implementation:

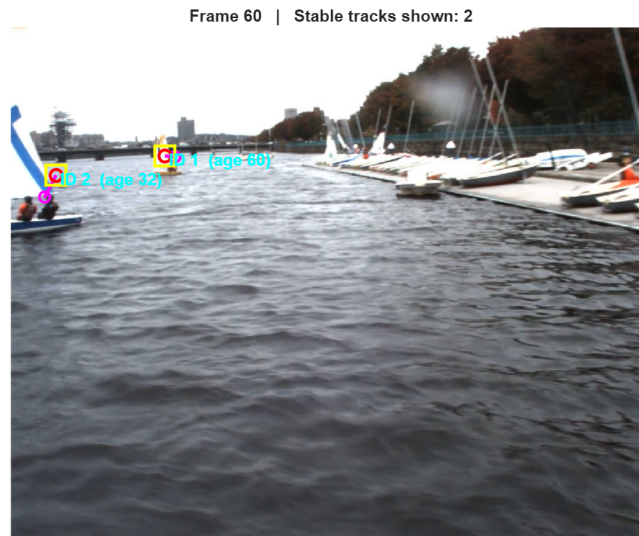


Figure 4.7: Centroid locations in red highlighted with yellow box, cyan text is the track ID (correct this time) and purple line indicates the velocity of the ship.

To observe all video outputs from each iteration of the tracker, please download the onedrive folder listed in the github README.

Chapter 5

Discussion and Results

The tracking algorithm correctly tracks the ships while they are in frame, and not hidden by an occlusion. When they leave the frame or are hidden, they are assigned a new track ID upon re-entry. This is something that requires resolution through the incorporation of other data sources.

GPS data may prove to be relatively difficult to use for this task. The main reason is the fact that the camera where the video footage comes from is placed on a ship that moves. This means that homography (transforming pixel coordinates to global coordinates) would be near impossible, as homography requires at least four in frame location's global coordinates to be known.

A solution could be to apply the same tracking and detection algorithms on the left and center videos, and utilize the radar data to associate ID and track between the videos.

5.1 A note for whoever is potentially taking this on

It is really easy to get swept away by MATLAB/PyTorch's libraries of built in functions that most probably will do a much better job than you could. My advice is to at least try to understand some machine learning basics before going headfirst into a premade function.

For instance with the tracker section of this project, I first tried to figure it out and get something working on my own side, and with my knowledge of Kalman filters I tried to get it to work. Only after tinkering for a while and realizing the underlying issues, did I change to trackerGNN. Moreover, trackerGNN didn't even work well on its own, and I had to modify it's usage to get it to behave adequately.

The next chapter of this report (Resources) contains a list of things I believe one should go through to best understand the task and my MATLAB documents. Please skim through them.

Chapter 6

Resources

6.1 MATLAB resources

A solid start on working with CV/ML/DL tasks is to do the following MATLAB online mini-courses, they helped me greatly for this project:

1. Image Processing Onramp
2. Deep Learning Onramp
3. Computer Vision Onramp
4. Object Detection with Deep Learning

These give you a solid introduction to video file manipulation in MATLAB, frame extraction and augmentation, visualization, transfer learning, YOLOX, and tensor maths.

Other MATLAB resources that were helpful and not covered by the above courses were the utilization of trackerGNN.

6.2 ML/DL resources

A solid introduction to ML/DL is Stanford's CS229 and CS231.

6.3 Alternate data sets/next steps

A data set that could be used for just ship detection is the BONK data set. Here they have 6D pose estimation models for each degree of freedom for the ships, but have no other associated data besides video. They have immovable cameras, and hence homography is possible. They do this via two methods:

1. Projecting a plane tangential to the earth, where the plane touches the earth at the water surface and hence can approximate the water to be flat, and the water and plane and earth share the same normal vector.
2. Use principal component analysis to find the directions of maximum variance for a set of 3D points with orthogonal vectors v_1 , v_2 and v_3 . If the points are approximately planar (as they are here), almost all variance lies in the first 2 normal vectors and hence v_3 is approximately normal to the plane, and thus can be discarded to create a plane spanned by v_1 and v_2 .

Thus for the 3D set of points k_{3D} one can project these points onto 2D with a projection matrix 'A' defined as:

$$k_{2D} = A^T k_{3D}. \quad (6.1)$$

As all the relevant points are now projected onto a plane, homography can be conducted. The homography matrix 'H' is approximated by minimizing the representation error when mapping global coords on the interim plane to known pixel coords. H is defined as the map from global interim plane coordinates 'x2' to the image pixel coordinates 'x1':

$$x_2 = H x_1 \quad (6.2)$$

I believe this method above could be manipulated to fit our current dataset, with some major adjustments. First of all their camera doesn't move, making velocity predictions and this homography method relatively trivial.

An interesting issue with homography is that it assumes 2 things:

1. The scene is approximately flat (almost always true for marine datasets, ocean approx flat).
2. The camera only pans under ROTATION not translation (NOT present in this dataset).

This second item is the major issue here. It means that the depth every object detected is changes in each frame. This is an issue because we wish to find a single homography matrix H which applies to the scene. The reason this fails when depth changes is due to where H comes from [5]:

$$\mathbf{x}' \sim K \left(R + \frac{\mathbf{t} \mathbf{n}^\top}{d} \right) K^{-1} \mathbf{x}$$

Due to the dependency on depth 'd', there will be induced parallax. Instead of utilizing homography methods as discussed above, perhaps one could compute the fundamental matrix F, which tells you 'given a point in image 1, which line must it lie upon in image 2'. In other words, for a 3D point in real space X is imaged in image 1 as x, then upon which line in image 2 must x' exist, shown via the relation [6]:

$$x'^T F x = 0 \quad (6.3)$$

To calculate F, we need to know information about our camera, which is given in the Sea Lab database.

Read the document [6], I believe F to be able to be calculated through the use of the 8 point algorithm, noted by equations 10 - 14 in the above document.

The final chapter of this report is a short literature review on pre-existing data fusion techniques incorporating image and AIS data.

Chapter 7

Literature Review on AIS-Image Data Fusion

7.1 Transformation Methods

The core problem of fusing AIS data with image data is defining a transformation that takes image space coordinates ((x,y) pixel locations) to real world geographic coordinates (longitude and latitude (u,v)). A pipeline for detection-message association is often:

1. Detect ships using some deep learning algorithm (YOLO family or other).
2. Convert the bounding box output to a single pixel (x,y) location, often chosen as the geometric centroid of the bounding box.
3. Using some transformation method, convert the (x,y) pixel location to real world (u,v) coordinates.
4. Apply some matching algorithm to associate an AIS message, and hence ship identity, heading etc to the ship located at location (u,v).

Hence it is critical that the location transformation from $(x, y) \rightarrow (u, v)$ is accurate, such that AIS message association is not random.

The first step is vessel detection. This does not necessarily have to be done through the use of deep learning convolutional neural networks, but other machine learning methods could be utilized, such as support vector machines, k-means clustering or aggregated channel features. Although the CNN's are more complicated than the other methods, pretrained networks have been created for download, such as the YOLOVx family. For vessel detection, both [7] and [8] utilized members of the YOLOVx family. These pretrained networks are easy to work with, as you have the architecture given to you and all that needs to be conducted is transfer learning. IBM defines transfer learning as 'a machine learning technique in which knowledge gained through one task or dataset is used to improve model performance on another related task or different dataset'. In the programs discussed within this document, this process was also conducted. The process of transfer learning is loading in the pretrained network, and then retraining it on a new data set. Other adjustments to specific layers can also be conducted, such as tweaking the early layers such that the model expects images of mxnx3 size as opposed to its pretrained notion of kxhx3 or even kxhx1 if the images are grayscale.

There are many transformation methods available, and in this document there will be discussion and analysis of three major methods, homography, perspective-n-point and interpolation. These are the methods discussed in the articles [7] and [8].

7.1.1 Homography

First, let's discuss what homography is and the requirements thereof. Homography is the process of transforming elements of 3-space and compressing them to 2-space. Generally, homography is defined a 3x3 invertible non-singular matrix H that maps a 2D plane to itself, ie $H : P^2 \rightarrow P^2$ [9].

This is what needs to be conducted for mapping latitude and longitude coordinates to pixel locations, and vice versa. The issue lies in the fact that the image coordinates are clearly 2D but real world locations are obviously 3D. Technically all these points can be written in 3D, ideally we wish to have vectors $(x,y,1)$ and $(u,v,1)$.

The requirements for homography (ie the existence of a unique homography map H) are as follows:

1. All observed 3D points lie on a single plane Π (such as the sea surface...). Mathematically this is shown via the relation:

$$\mathbf{x}' \sim \mathbf{K} \left(\mathbf{R} + \frac{\mathbf{t}\mathbf{n}^\top}{d} \right) \mathbf{K}^{-1}\mathbf{x}$$

where $\mathbf{x}$ and $\mathbf{x}'$ are the locations in each image, $\mathbf{K}$ is the cameras intrinsic matrix (discussed later), $\mathbf{R}$ is a rotation matrix, $\mathbf{t}$ is the translation vector between the 2 camera centers, $\mathbf{n}$ is the unit normal to the observed 3D plane Π and finally d is the orthogonal distance from the first camera's center to the plane (scalar) [10].

2. The cameras can only move under pure rotation, in other words homography ARISES from pure camera rotation, generalized by the following:

$$\mathbf{H} = \mathbf{K}\mathbf{R}\mathbf{K}^{-1}$$

Proof:

Given no translation, this implies $\mathbf{t} = \mathbf{0}$, hence:

$$\mathbf{x}' \sim \mathbf{K} \left(\mathbf{R} + \frac{\mathbf{t}\mathbf{n}^\top}{d} \right) \mathbf{K}^{-1}\mathbf{x} = \mathbf{K}\mathbf{R}\mathbf{K}^{-1}\mathbf{x}$$

To estimate the homography matrix, we start with $\mathbf{x}' \sim H\mathbf{x}$. Rewriting this element by element and using the following notation:

$$\begin{aligned} \mathbf{h} &= \left(H_{11}, H_{12}, H_{13}, H_{21}, H_{22}, H_{23}, H_{31}, H_{32}, H_{33} \right)^\top \\ \mathbf{a}_x &= \left(-x_1, -y_1, -1, 0, 0, 0, x'_2x_1, x'_2y_1, x'_2 \right)^\top \\ \mathbf{a}_y &= \left(0, 0, 0, -x_1, -y_1, -1, y'_2x_1, y'_2y_1, y'_2 \right)^\top \end{aligned}$$

We can define the following homogenous system problem:

$$\mathbf{A}\mathbf{h} = \mathbf{0}$$

where

$$\mathbf{A} = \begin{pmatrix} \mathbf{a}_{x_1}^\top \\ \mathbf{a}_{y_1}^\top \\ \vdots \\ \mathbf{a}_{x_N}^\top \\ \mathbf{a}_{y_N}^\top \end{pmatrix}$$

To calculate $\mathbf{h}$ (hence the elements of $\mathbf{H}$) we can perform least squares. Unfortunately because it is not inhomogenous, we cannot simply calculate the pseudoinverse of $\mathbf{A}$, but instead can utilize the singular value decomposition (SVD), rewriting $\mathbf{A}$ as:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top = \sum_{i=1}^9 \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

We can then take the right singular vector of $\mathbf{A}$ corresponding to the smallest singular value (σ_9), and is thus our solution $\mathbf{h}$. For more reading on this derivation, please read [11].

BONK dataset (Fabian Holst et al)

Homography is a method utilized by both [7] and [8]. The goal of the BONK dataset creators Fabian Holst et al ([7]) was to create a dataset for 3D bounding boxes around detected ships. These bounding boxes indicate all 6 degrees of freedom of the detected ship (3 rotational and 3 translational). To do this, they fused monocular RGB camera data with AIS messages. Their pipeline is shown in the image below:

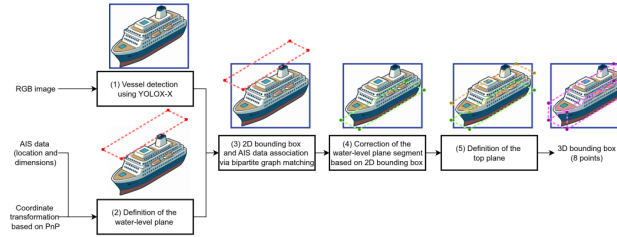


Figure 7.1: BONK pipeline for acquiring 3D pose estimations from image and AIS data fusion.

The important step for this discussion is step 2: coordinate transformation. Their first step was keypoint selection, a crucial feature of all homography based approaches. These keypoints are the points which must be coplanar for $\mathbf{H}$ to exist. They ensured this using 2 methods, the first through by determining the plane tangential to the water's surface and sharing the same normal to the earth at that point, and another through PCA and removing the dimension with the least variance.

After the planar projection, they were able to perform homography by minimizing the reprojection error caused when mapping the interim plane geographic coordinates to the pixel locations.

Emre Gulsoylu et al

Emre Gulsoylu et al ([8]) utilized the YOLO family for vessel detection, and also utilized homography to match AIS messages with detected ships.

They did not have access to camera parameters (and hence the K matrix), so instead of coplanar images they decided to use an image and a map of the same area to generate H . Their setup is shown below:



Figure 7.2: Pipeline utilized by Emre Gulsoylu et al to associate ship detections with AIS information, highlighting keypoint locations on the map and image.

For scaling, they decided to incorporate w as the third dimension for their map (lon, lat, w) coordinates to account for scaling, and appended 1 to their $(x,y,1)$ image coordinates to ensure there was no dimension mismatch. For their fixed cameras, they detected keypoints and used Google Earth to associate these with real world coordinates, and hence estimated H .

7.1.2 Perspective-n-point (used only by BONK)

For their second method of transforming pixel locations to geographic coordinates. They used a pinhole camera model, defined by the following equation:

$$\mathbf{x} = \mathbf{K} \mathbf{\Pi}^c \mathbf{T}_w \mathbf{X}_w$$

This no longer requires keypoints to be projected onto an interim plane, as it allows the full use of 3 space. It is easiest to understand this equation right to left: $\mathbf{X}_w$ is the real world coordinate vector. It is then projected, using the cameras extrinsic parameters, into a frame of reference relative to the camera by the matrix ${}^c\mathbf{T}_w$, which holds the cameras real world position vector as well as its rotation. $\mathbf{\Pi}$ projects onto a plane where $\mathbf{K}$ (the intrinsic camera parameters) containing FOV, AR etc finally

transforms it into the image pixel location $\mathbf{x}$.

7.1.3 Interpolation (Emre Gulsoylu et al)

Emre Gulsoylu et al utilized a dataset where the images in their dataset were almost perfectly aligned with the horizon and hence their X-axis and azimuth (deg) had a linear relationship. Hence every pixel on the image X-axis could be mapped to an azimuth angle, where the distance could hence be estimated via interpolation.

7.2 Association Methods

Fabian Holst et al utilized bipartite graph matching to associate AIS messages with detected vessel locations. Their formulation and weight derivation for each node was relatively complex due to their end goal of creating a 3D bounding box instead of simple identification. The underlying idea was to set the matching cost based on the Euclidean distance between their water level segment and their 2D bounding box.

Emre Gulsoylu et al utilized velocity interpolation and the assumption that the vessels moved at approximately constant speed, as well as defining a region of interest filter on AIS messages for the specific region, and matched the AIS messages via a k-dimensional tree (k-th nearest neighbour).

Bibliography

- [1] Mathworks, “Kalman filtering.” [Online]. Available: <https://www.mathworks.com/help/control/ug/kalman-filtering.html>
- [2] Y. Guo, “Asynchronous trajectory matching-based multimodal maritime data fusion for vessel traffic surveillance in inland waterways.” [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10159572>
- [3] S. Grant, “Auv lab philos – two sailboats – close approach.” [Online]. Available: <https://seagrants.mit.edu/auvlab-datasets-marine-perception-2-3/#toggle-id-4>
- [4] Mathworks, “trackergnn multi-sensor, multi-object tracker using gnn assignment.” [Online]. Available: <https://www.mathworks.com/help/fusion/ref/trackergnn-system-object.html>
- [5] T. Riklin-Raviv, “Prior-based segmentation by projective registration and level sets.” [Online]. Available: https://www.researchgate.net/publication/4193840_Prior-based_segmentation_by_projective_registration_and_level_sets
- [6] K. Hata, “Cs231a course notes 3: Epipolar geometry.” [Online]. Available: https://web.stanford.edu/class/cs231a/course_notes/03-epipolar-geometry.pdf
- [7] F. H. et al, “Fusing monocular rgb images with ais data to create a 6d pose estimation dataset for marine vessels.” [Online]. Available: <https://arxiv.org/pdf/2508.14767>
- [8] E. G. et al, “Image and ais data fusion technique for maritime computer vision applications.” [Online]. Available: <https://arxiv.org/pdf/2312.05270>
- [9] E. Dubrofsky, “Homography estimation.” [Online]. Available: https://www.cs.ubc.ca/sites/default/files/2022-12/Dubrofsky_Elan.pdf
- [10] R. Hartley and A. Zisserman, “Multiple view geometry in computer vision second edition.” [Online]. Available: http://www.r-5.org/files/books/computers/algo-list/image-processing/vision/Richard_Hartley_Andrew_Zisserman-Multiple_View_Geometry_in_Computer_Vision-EN.pdf
- [11] D. Kriegman, “Homography estimation.” [Online]. Available: https://cseweb.ucsd.edu/classes/wi07/cse252a/homography_estimation/homography_estimation.pdf?utm_source=chatgpt.com